



AI with Rav

Learn AI the way you actually think.



DAY 2 of 30

PYTHON

The Python trap that breaks quietly

WHAT YOU'LL LEARN TODAY

- > The bug that hides for months
- > Why Python behaves this way
- > The one-line fix
- > How to spot it in review



The Python trap that breaks quietly

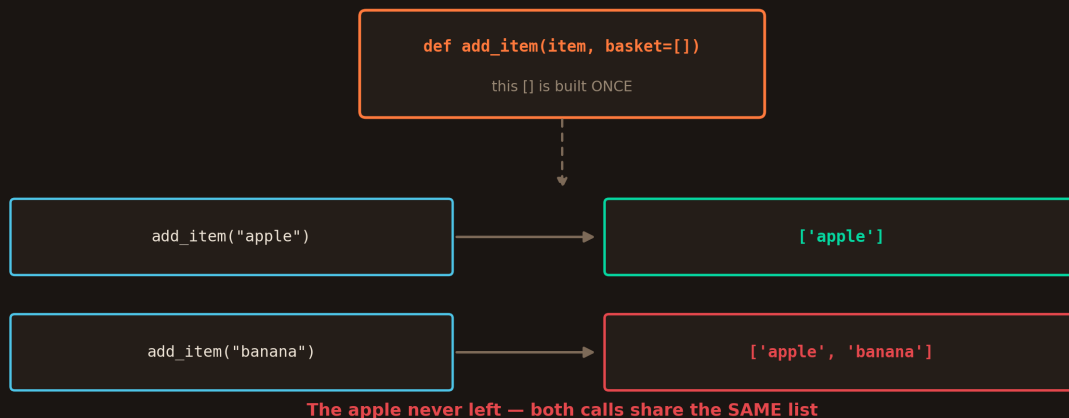
In One Line: A default list in a Python function is created ONCE — not every time you call it.

The code that looks fine

```
def add_item(item, basket=[]):  
    basket.append(item)  
    return basket  
  
print(add_item("apple"))    # ['apple']        looks right  
print(add_item("banana"))  # ['apple', 'banana']  what?
```

The second call had nothing to do with the first. Yet the apple is still there.

ONE LIST, CREATED ONCE, SHARED BY EVERY CALL



The default list is created once, when the function is defined — every call shares that same list

Why this happens

Python builds the default value **when it reads the def line** — once, at import time. Not on each call.

So every call that does not pass a basket shares the *same* list. Forever. One basket for the whole programme.

The fix is one line



```
def add_item(item, basket=None):  
    if basket is None:  
        basket = []  
    basket.append(item)  
    return basket
```

None is immutable, so there is nothing to accumulate. A fresh list is made on every call that needs one.

This bites hardest in long-running services. Locally it looks fine; in production the list grows for days.

Rule: never put a list, dict or set in a default argument. Use None and build it inside.

